

# Detecção de Placas Veiculares com MobileNetV1 e Detector por Grade Embarcado em ESP32-S3

Carlos Vinícius dos Santos Mesquita, Alisson Jaime Sales Barros,  
Victor Guedes Alves Teixeira e Kaique Ferreira Braga Silva  
TI0147 — Fundamentos de Processamento Digital de Imagens  
Professor: Paulo Cesar Cortez  
Universidade Federal do Ceará

**Resumo**—Este trabalho apresenta um sistema completo de detecção de placas veiculares baseado na arquitetura MobileNetV1 com uma cabeça de detector por grade (saída espacial  $7 \times 7 \times 5$ ), treinado em TensorFlow/Keras e embarcado em uma placa ESP32-S3 com câmera OV5640. O modelo foi quantizado em INT8 e convertido para TensorFlow Lite, viabilizando a inferência diretamente no microcontrolador. Avaliamos o sistema em três modalidades (imagem, vídeo e captura em tempo real com a ESP32-CAM), comparamos o desempenho entre PC e dispositivo embarcado, e aplicamos pós-processamento com filtragem por confiança e segmentação dos caracteres. Os resultados, obtidos sobre cinco execuções, mostram IoU médio de  $0,731 \pm 0,005$  no conjunto combinado e *recall* de 0,922, com a quantização INT8 preservando a qualidade (IoU praticamente idêntico ao modelo Keras). A inferência custa  $\sim 4\text{--}7$  ms no PC contra  $\sim 117$  s na ESP32-S3, evidenciando o gargalo do hardware embarcado. O acréscimo de imagens negativas reduziu os falsos positivos a zero no limiar 0,95.

**Index Terms**—detecção de objetos, MobileNetV1, TensorFlow Lite, sistemas embarcados, ESP32, quantização INT8, processamento digital de imagens.

## I. INTRODUÇÃO

A detecção de objetos é uma tarefa central da Visão Computacional, consistindo em localizar objetos de interesse por meio de *bounding boxes*. Sua execução em sistemas embarcados, contudo, é desafiadora: dispositivos como a ESP32 possuem forte restrição de memória e processamento. Arquiteturas da família MobileNet, baseadas em convoluções separáveis em profundidade (*depthwise separable convolutions*), foram projetadas para esse cenário, reduzindo parâmetros e custo computacional.

Este trabalho desenvolve, treina e embarca um detector de placas veiculares usando MobileNetV1 com uma cabeça de detecção por grade, e o avalia nas três modalidades exigidas (imagem, vídeo e tempo real). Além do embarcamento na ESP32-S3, o mesmo modelo é executado em PC para comparação de desempenho, e o sistema é complementado por pós-processamento (filtragem por confiança e segmentação dos caracteres da placa).

## II. BASE DE DADOS

Foram utilizadas duas bases de detecção de objetos de classe única (*plate*), com anotações no formato YOLO (centro, largura e altura normalizados): **Base 1**, fornecida pelo professor, e **Base 2**, a base pública *Brazil Plates Detector*

Tabela I  
DIVISÃO DA BASE UTILIZADA (UMA CLASSE: *plate*).

| Conjunto               | Treino | Validação |
|------------------------|--------|-----------|
| Base 1 (professor)     | 350    | 60        |
| Base 2 (Brazil Plates) | 200    | 12        |
| Mix ponderado          | 972    | 72        |
| Negativas (sem placa)  | 12     | 4         |

(Roboflow, licença CC BY 4.0), escolhida pela aderência ao problema. As bases foram combinadas com ponderação da Base 2 para reforçar exemplos difíceis. A Tabela ?? resume a divisão efetivamente utilizada. Adicionalmente, incorporamos **16 imagens negativas** (cenar sem placa) para reduzir falsos positivos.

## III. ARQUITETURA E DETECTOR POR GRADE

A arquitetura é a **MobileNetV1** com fator de largura (*width multiplier*)  $\alpha = 0,5$  e entrada  $224 \times 224 \times 3$ . Sobre o *backbone* **pré-treinado na ImageNet** adicionamos uma **cabeça de detecção por grade** que produz uma saída espacial  $7 \times 7 \times 5$ : a imagem é dividida em uma grade  $7 \times 7$  e cada célula prevê uma confiança de objetos (*objectness*) e os quatro parâmetros da caixa ( $c_x, c_y, w, h$ ). A detecção final corresponde à célula de **maior confiança**, e a caixa global é reconstruída por  $c_x = (\text{col} + c_x^{\text{local}})/7$  e  $c_y = (\text{lin} + c_y^{\text{local}})/7$ . Essa formulação resolveu o problema de localização da região correta da placa observado em abordagens de regressão direta de uma única caixa. A escolha do MobileNetV1  $\alpha = 0,5$  equilibra precisão e tamanho, essencial para o embarcamento.

## IV. METODOLOGIA

### A. Pré-processamento

As imagens são redimensionadas para  $224 \times 224$  e normalizadas para o intervalo  $[-1, 1]$  por  $x_{\text{norm}} = x/127,5 - 1$  (Figura ??). Em PDI é usual operar apenas sobre a **luminância** (imagens de um único canal); a MobileNetV1, porém, exige entrada de três canais. Para contornar essa limitação da arquitetura escolhida e ainda assim avaliar o comportamento clássico em tons de cinza, definimos o modo **gray3**, em que o canal de luminância é replicado nos três canais de entrada — ganhando robustez a variações de cor entre bases e câmeras —, além do modo **RGB** convencional. Os dois modos foram

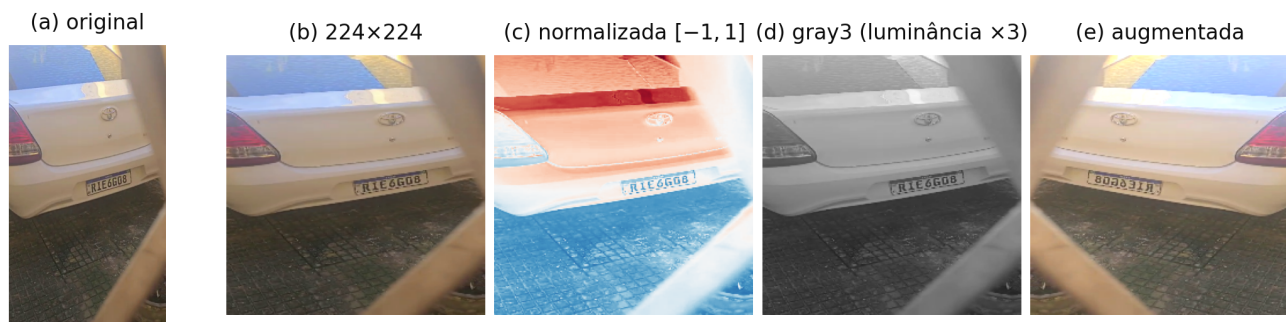


Figura 1. Pré-processamento passo a passo: (a) imagem original; (b) redimensionamento para  $224 \times 224$ ; (c) normalização para  $[-1, 1]$  (visualizada em falsa cor); (d) modo gray3 (luminância replicada em três canais); (e) exemplo de *Data Augmentation* (espelhamento + brilho).

avaliados e permanecem selecionáveis em tempo de execução no *frontend*. Durante o treino aplicamos *Data Augmentation*: espelhamento horizontal, variações de brilho/contraste, ruído e desfoque (Figura ??e).

### B. Treinamento

O treino emprega o otimizador Adam em duas fases (*transfer learning* seguido de *fine-tuning* da cabeça): 12 e 50 épocas, *batch* 8, com *ReduceLROnPlateau* e *EarlyStopping*. A função de perda combina quatro termos com pesos fixos,

$$\mathcal{L} = 2\mathcal{L}_{\text{conf}}^{\text{obj}} + 0,25\mathcal{L}_{\text{conf}}^{\text{noobj}} + 8\mathcal{L}_{\text{box}} + 3\mathcal{L}_{\text{GloU}}, \quad (1)$$

em que  $\mathcal{L}_{\text{conf}}^{\text{obj}}$  e  $\mathcal{L}_{\text{conf}}^{\text{noobj}}$  são o erro quadrático da confiança, médio nas células *com* e *sem* objeto (o peso menor das células vazias evita que a maioria negativa domine o gradiente);  $\mathcal{L}_{\text{box}}$  é a *Smooth-L1* dos parâmetros locais ( $c_x, c_y, w, h$ ) na célula positiva; e  $\mathcal{L}_{\text{GloU}}$  é a perda GIoU da caixa global reconstruída a partir da grade, que maximiza a sobreposição real entre predição e anotação. Cada configuração foi executada **cinco vezes**, reportando-se média e desvio padrão (Seção ??). O ambiente de treino foi o Google Colab (TensorFlow 2.20.0, Python 3.12, GPU NVIDIA Tesla T4).

### C. Pós-processamento

Aplicamos **filtragem por confiança**: a detecção só é considerada válida quando a confiança ultrapassa um limiar operacional (projetado em 0,95 e recalibrável em campo; Seção ??) e se mantém por **2 frames consecutivos**, reduzindo disparos espúrios. Em seguida, realizamos a **segmentação dos caracteres** da placa (obrigatória) por técnicas clássicas de OpenCV, ilustradas etapa a etapa na Figura ??: (i) recorte da ROI prevista; (ii) equalização local de contraste (CLAHE); (iii) transformação *BlackHat*, que realça estruturas escuras (caracteres) sobre fundo claro (placa); (iv) **limiarização de Otsu** sobre o resultado — o método escolhe automaticamente o limiar que maximiza a variância entre as classes fundo/caractere do histograma (Figura ??d) — combinada com limiar adaptativo; e (v) limpeza morfológica com filtragem de componentes conectados por área e proporção. A Figura ?? mostra o resultado sobre um quadro de vídeo e a Figura ??

apresenta exemplos adicionais em fotografias reais capturadas para o trabalho.

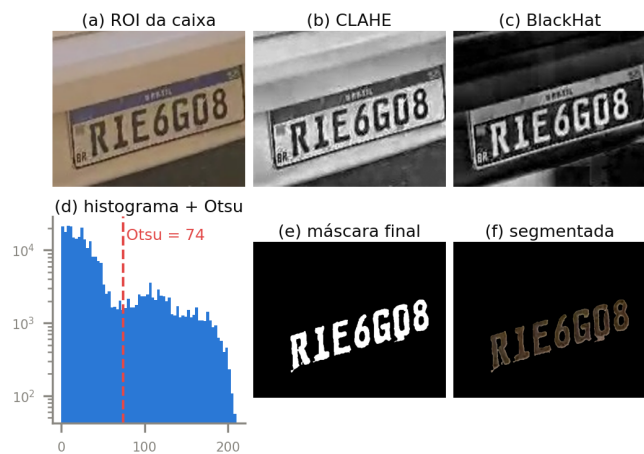


Figura 2. Etapas da segmentação: (a) ROI recortada da caixa prevista; (b) CLAHE; (c) BlackHat; (d) histograma do BlackHat com o limiar de Otsu selecionado (linha tracejada); (e) máscara após limpeza morfológica; (f) caracteres segmentados.



Figura 3. Pós-processamento: ROI da placa, máscara de segmentação e caracteres segmentados (OpenCV).

### D. Conversão e Quantização para a ESP32

O modelo Keras foi convertido para TensorFlow Lite em três variantes (Float32, *dynamic* e INT8). A quantização **INT8 pós-treino** foi escolhida para o embarcamento. O arquivo *.tflite* (2,35 MB) foi convertido em um *array* C++ (*xxd -i*), gerando um cabeçalho de **2.468.584 bytes**. No firmware

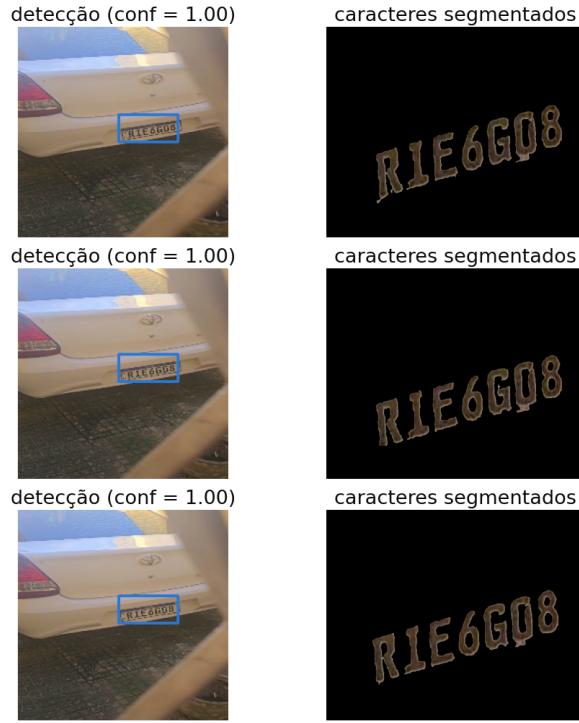


Figura 4. Exemplos adicionais de detecção e segmentação em três fotografias reais (conjunto *imagens\_teste*).

(Arduino/ESP-IDF, biblioteca *TensorFlowLite\_ESP32*), o *tensor arena* (3 MB) é alocado na PSRAM e uma tabela de partições personalizada de 5 MB acomoda o aplicativo (3,74 MB de *flash*). A placa é uma **ESP32-S3** (16 MB *flash*, 8 MB PSRAM OPI, CPU a 240 MHz) com câmera **OV5640**.

#### E. Condições Experimentais

O **treinamento e a avaliação das métricas** foram realizados no Google Colab (TensorFlow 2.20.0, Python 3.12, GPU NVIDIA Tesla T4). Os **testes em PC** (execução LiteRT/TFLite) usaram um notebook com CPU Intel Core i7-14700HX, 16 GB de RAM, Windows 11 (64 bits), Python 3.14. O **dispositivo embarcado** é uma ESP32-S3 (16 MB *flash*, 8 MB PSRAM OPI, CPU a 240 MHz) com câmera OV5640, usando a biblioteca *TensorFlowLite\_ESP32* (Arduino-ESP32 3.3.8 / ESP-IDF 5.5).

### V. RESULTADOS

#### A. Desempenho de detecção (cinco execuções)

A Tabela ?? apresenta as métricas médias e desvios das cinco execuções. O modelo atinge *recall* elevado na Base 1 e no conjunto combinado; o desempenho menor na Base 2 reflete sua maior dificuldade e menor tamanho. O *score balanceado* indicado na tabela é a média simples dos IoU médios das duas bases,  $(IoU_{B1} + IoU_{B2})/2$  — critério usado para selecionar o modelo final sem favorecer a base maior.

Tabela II  
MÉTRICAS EM CINCO EXECUÇÕES (MÉDIA ± DESVIO).

| Métrica                   | Base 1        | Base 2        | Mix           |
|---------------------------|---------------|---------------|---------------|
| IoU médio                 | 0,768 ± 0,002 | 0,543 ± 0,030 | 0,731 ± 0,005 |
| Recall ( $IoU \geq 0,5$ ) | 0,973 ± 0,013 | 0,667 ± 0,075 | 0,922 ± 0,014 |
| F1@0,5                    | 0,983         | 0,750         | 0,944         |

Score balanceado (média dos IoU de B1 e B2): 0,655 ± 0,015.

Tabela III  
TEMPO DE INFERÊNCIA, TAMANHO E IOU (CONJUNTO MIX).

| Plataforma / modelo           | Tempo         | Tamanho | IoU   |
|-------------------------------|---------------|---------|-------|
| Keras (PC, float)             | ~96,9 ms      | n/d     | 0,736 |
| TFLite Float32 (PC)           | ~6,5 ms       | 8,75 MB | 0,736 |
| TFLite INT8 (PC, LiteRT)      | ~4–7 ms       | 2,35 MB | 0,732 |
| <b>TFLite INT8 (ESP32-S3)</b> | <b>~117 s</b> | 2,35 MB | n/d   |

#### B. Comparação PC vs. ESP32 e validação da quantização

A Tabela ?? compara plataformas e a Figura ?? resume os tempos em escala logarítmica. A quantização INT8 **preservou a qualidade** (IoU praticamente idêntico ao Keras), reduzindo o modelo de 8,75 MB (Float32) para 2,35 MB. No PC a inferência custa poucos milissegundos; na ESP32-S3, ~117 s por *frame*, cerca de  $1,7 \times 10^4$  vezes mais lenta. O gargalo decorre dos *kernels* de referência da biblioteca TFLite Micro e do acesso à PSRAM, não do modelo em si.

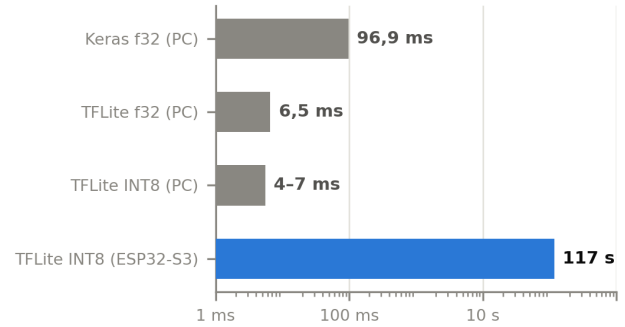


Figura 5. Tempo de inferência do mesmo modelo em cada plataforma (escala logarítmica). O arquivo INT8 é idêntico no PC e na ESP32-S3.

#### C. Efeito das imagens negativas

A Tabela ?? mostra os falsos positivos sobre imagens sem placa em função do limiar. No limiar adotado (0,95) não há falsos positivos, e os exemplos positivos não foram degradados (confiança média ~ 0,99). Medições na própria ESP32-S3 confirmaram o ganho: cenas sem placa caíram de ~ 0,62 (modelo antigo) para 0,42–0,58, enquanto uma placa real bem enquadrada atinge 0,98–0,99.

#### D. Avaliação nas três modalidades

**Imagem.** A Figura ?? mostra a detecção em uma imagem: a caixa é posicionada corretamente sobre a placa e o mapa de confiança da grade destaca a célula correta.

Tabela IV  
FALSOS POSITIVOS EM IMAGENS SEM PLACA POR LIMIAR.

| Limiar           | 0,50 | 0,90 | 0,95       | 0,98 |
|------------------|------|------|------------|------|
| Falsos positivos | 4/4  | 1/4  | <b>0/4</b> | 0/4  |

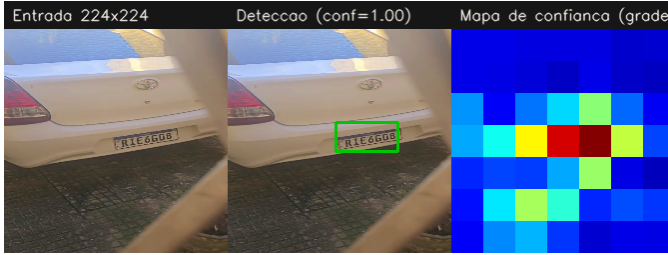


Figura 6. Detecção em imagem: entrada, caixa prevista (confiança  $\approx 1,0$ ) e mapa de confiança da grade  $7 \times 7$ .

**Vídeo.** O modelo foi executado quadro a quadro sobre uma sequência de vídeo (Figura ??), atingindo confiança de até 0,996 no melhor *frame*, com taxa de processamento de centenas de FPS no PC.



Figura 7. Detecção em vídeo (melhor *frame*, confiança 0,996).

**Tempo real.** A captura ao vivo com a OV5640 acoplada à ESP32-S3 produziu detecção embarcada com confiança 0,97, disparando o envio automático da imagem anotada (Telegram). A Figura ?? ilustra a correta *rejeição* de um quadro sem placa nítida (confiança abaixo do limiar).

A Figura ?? registra uma sessão completa de operação em campo (02/07): cada ponto é uma inferência embarcada

( $\sim 117$  s). Observa-se que a confiança com placa presente depende do enquadramento e da iluminação da cena — nessa sessão, o platô ficou em 0,82–0,92, abaixo dos 0,95–0,99 obtidos em cenas bem iluminadas. Por isso o limiar de disparo é tratado como **parâmetro de operação calibrável em campo** (entre 0,85 e 0,95, sempre acima da faixa sem placa de 0,42–0,58): com o limiar recalibrado para 0,85, dois *frames* consecutivos válidos dispararam o envio, e a fotografia anotada chegou ao Telegram **18 s** após a detecção. A exigência de 2 *frames* consecutivos manteve-se efetiva: oscilações isoladas abaixo do limiar zeraram a contagem sem gerar disparos espúrios.

## VI. DISCUSSÃO

A quantização INT8 mostrou-se vantajosa: reduziu o modelo em  $\sim 73\%$  sem perda mensurável de IoU, viabilizando o embarcamento. A principal limitação é o **tempo de inferência na ESP32-S3** ( $\sim 117$  s/frame), atribuível aos *kernels* de referência da TFLite Micro e à latência da PSRAM, e não ao tamanho do modelo, já que o mesmo arquivo roda em milissegundos no PC. As imagens negativas eliminaram os falsos positivos no limiar de operação. A qualidade da câmera OV5640 e o desfoque por movimento afetam a confiança, mitigados por bom enquadramento e pela exigência de detecção estável por múltiplos *frames*.

### A. Hipóteses para a latência de $\sim 2$ minutos por frame

Como o mesmo arquivo INT8 executa em milissegundos no PC, a lentidão está no ambiente de execução embarcado, não no modelo. As hipóteses, em ordem de impacto estimado, e as respectivas soluções são:

- 1) **Kernels de referência da TFLite Micro:** a biblioteca TensorFlowLite\_ESP32 usa implementações genéricas em C++, sem instruções vetoriais nem otimizações para o Xtensa LX7. *Solução:* migrar para o esp-tflite-micro oficial da Espressif com os *kernels* otimizados **ESP-NN**, que aceleram convoluções INT8 tipicamente em uma ordem de grandeza.
- 2) **Tensor arena na PSRAM:** os 3 MB de área de trabalho residem na PSRAM externa (barramento OPI), cuja latência é muito maior que a da SRAM interna; cada acesso a ativações intermediárias paga esse custo. *Solução:* reduzir o pico de memória (entrada menor, menos filtros) até a arena caber, ao menos parcialmente, na SRAM interna, ou elevar o *clock* da PSRAM.
- 3) **Custo computacional da entrada  $160 \times 160$ :** as primeiras camadas da MobileNetV1 dominam o número de multiplicações-acumulações. *Solução:* treinar com entrada  $96 \times 96$  ou aplicar *width multiplier*  $\alpha < 1$ , reduzindo o custo de forma quase quadrática com pequena perda de IoU.
- 4) **Execução em núcleo único a 240 MHz sem aceleração dedicada:** a ESP32-S3 não possui NPU, e a inferência ocupa um único núcleo. *Solução alternativa:* transmitir o *frame* da câmera para inferência em um servidor na rede local, mantendo o dispositivo apenas como captura.



## Operação em campo (02/07) — uma inferência a cada ~117 s

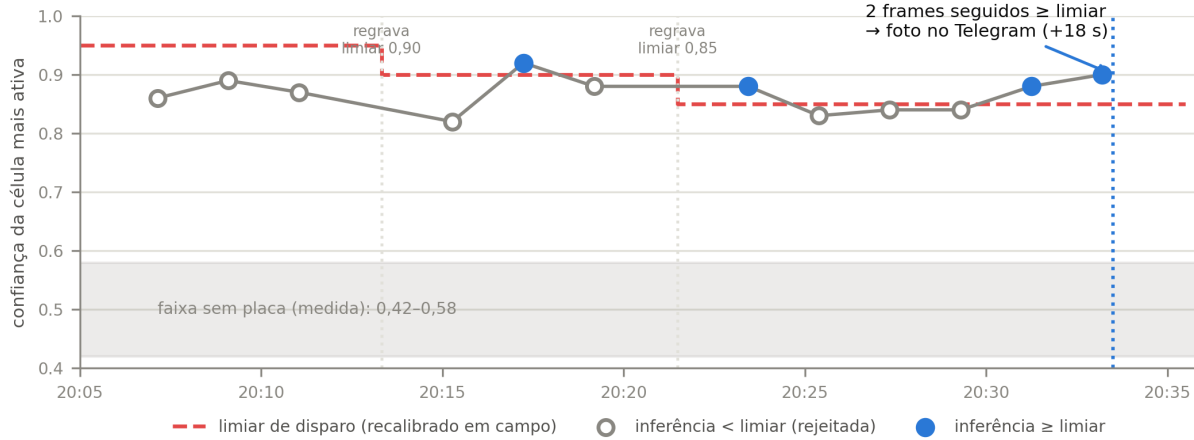


Figura 8. Sessão de operação em campo (02/07): confiança por inferência na ESP32-S3, limiar de disparo recalibrado em campo (linha tracejada), faixa sem placa medida (área sombreada) e disparo do Telegram após 2 *frames* consecutivos acima do limiar.



Figura 9. Rejeição correta: quadro sem placa nítida, confiança abaixo do limiar.

alternativamente, transmitir a captura da ESP32-CAM para inferência em servidor (*frontend* fluido); (iii) ampliar a base de negativas com fotos capturadas pela própria ESP-CAM; e (iv) reconhecimento óptico (OCR) dos caracteres já segmentados.

## REFERÊNCIAS

- [1] A. G. Howard *et al.*, “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv:1704.04861, 2017.
- [2] M. Abadi *et al.*, “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015. [Online]. Disponível: <https://www.tensorflow.org>
- [3] “TensorFlow Lite / LiteRT Guide.” [Online]. Disponível: <https://www.tensorflow.org/lite>
- [4] Roboflow, “Brazil Plates Detector Dataset.” [Online]. Disponível: <https://universe.roboflow.com/testes-bbb7m/brazil-plates-detector-zn2t0>
- [5] F. Chollet *et al.*, “Keras.” [Online]. Disponível: <https://keras.io>
- [6] Espressif, “ESP-IDF / ESP32-S3 Technical Reference.” [Online]. Disponível: <https://docs.espressif.com>

## VII. CONCLUSÃO E TRABALHOS FUTUROS

Desenvolvemos um detector de placas com MobileNetV1 e cabeça por grade, embarcado e funcional na ESP32-S3, avaliado nas três modalidades e comparado ao PC. A solução é viável e precisa, com a quantização preservando a qualidade. Como trabalhos futuros: (i) acelerar a inferência embarcada com *kernels* otimizados (ESP-NN) ou entrada menor ( $96 \times 96$ ); (ii)